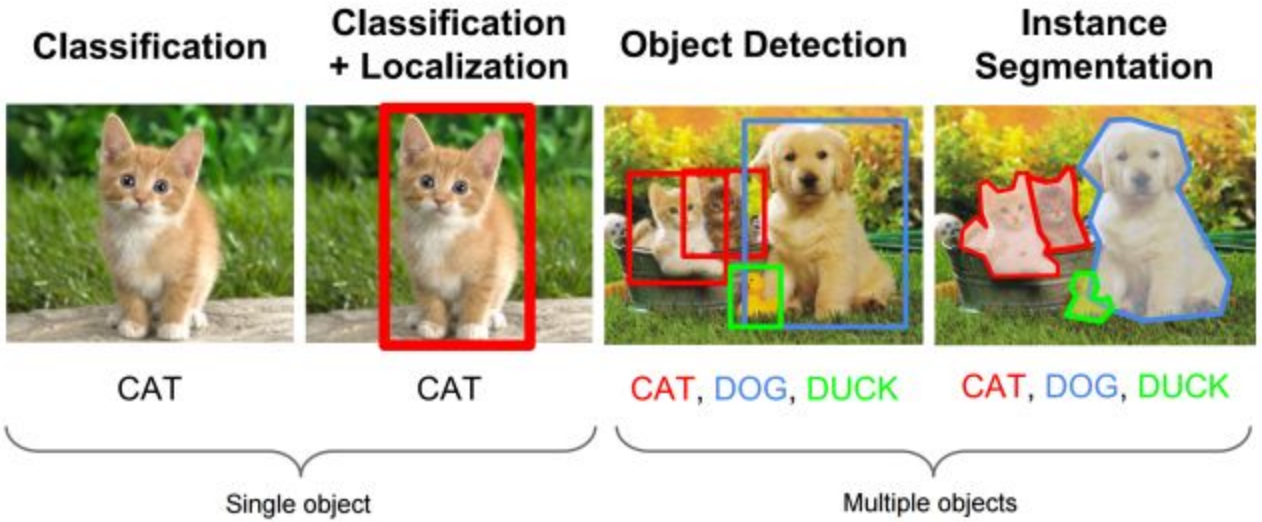
Image Classification .vs. Object Detection .vs. Instance Segmentation



객체 탐지(Object Detection) 시스템

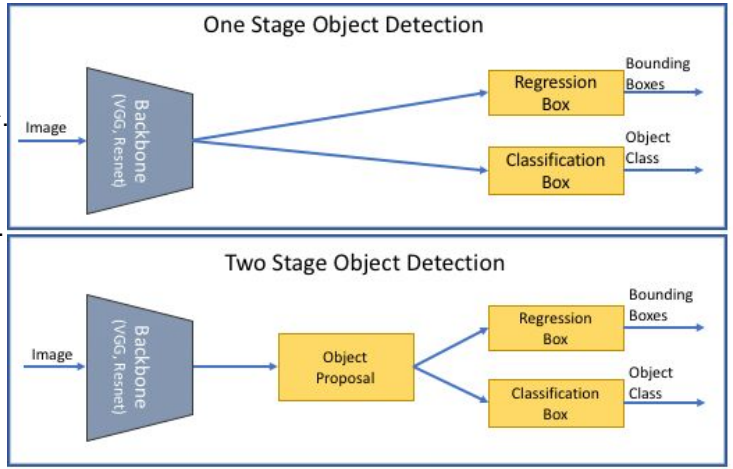
이미지나 비디오 내의 객체를 식별하고 위치를 추정하는 알고리즘 또는 모델을 의미. 객체 탐지에서 detector는 특정 객체가 이미지에 존재하는지 여부를 판단하고, 그 객체의 위치를 나타내는 bounding box를 생성하는 역할함. 이 과정은 크게 두 가지로 나누어짐

Classification (분류): 객체가 어떤 종류인지를 예측하는 단계. 예) 이미지 내의 어떤 객체가 사람인지, 고양이인지, 자동차인지 분류하는 작업을 수행
Localization (위치 추정): 객체가 이미지 안에서 어느 위치에 존재하는지를 판단. 각 객체에 대해 bounding box를 생성. 해당 객체가 포함된 영역을 사각형으로 표시

객체 탐지 모델은 이 두 가지 작업을 동시에 수행하는 방식으로, 분류와 위치 추정을 모두 처리함.
객체 탐지 모델이 생성한 bounding box는 객체의 위치를 정의하며, 모델은 각 박스가 어떤 객체에 해당하는지를 예측합니다.

2-Stage vs. 1-Stage Detector

객체 탐지(Object Detection) 모델은 크게 1-stage detector와 2-stage detector로 구분.
이 두 방식은 객체 탐지의 처리 방식과 속도, 성능에서 차이를 보이며 각각의 장단점이 있음.

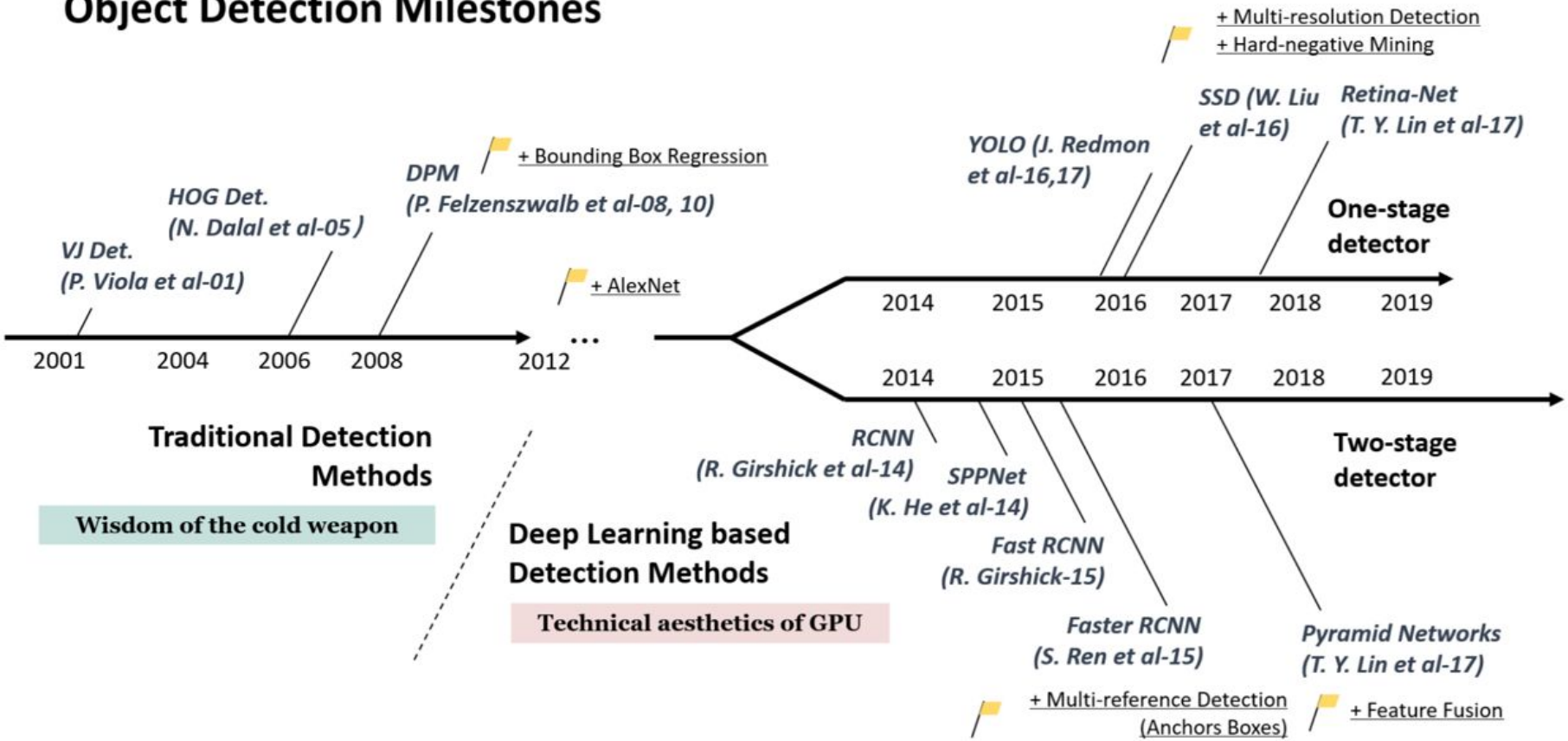


2-Stage Detector
먼저 이미지 내에서 객체가 있을 법한 **후보 영역을 추출**한 후, 그 **영역에 대해 분류를 수행**.
정확도는 높지만 처리 시간이 상대적으로 걸림. 대표모델 : Faster R-CNN

1-Stage Detector
anchor box와 CNN을 통한 동시 처리 방식으로, **이미지를 한 번에 처리하여 객체를 탐지**.
속도가 빠르지만 정확도는 상대적으로 낮을 수 있음. 대표모델 : YOLO

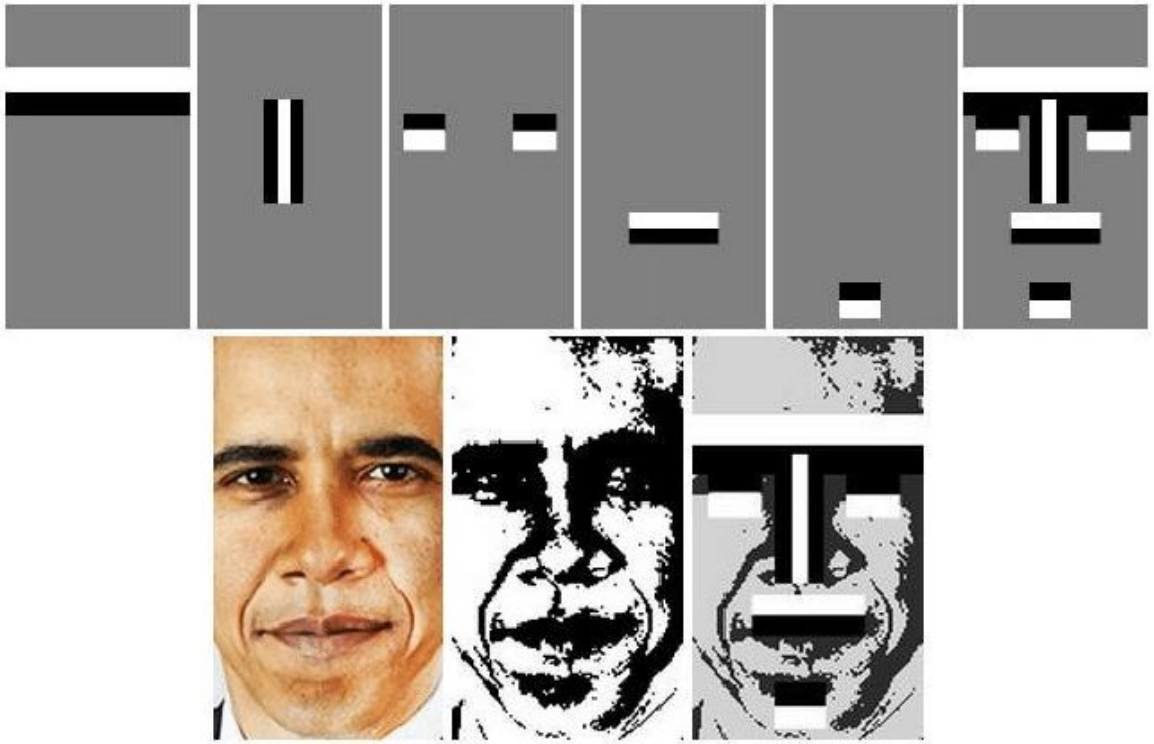
객체 탐지에서 detector는 객체를 찾아내고, 그 위치와 종류를 예측하는 핵심적인 역할을 수행

Object Detection Milestones



Viola

객체 감지 알고리즘의 첫번째 성공적 구현 사례 - 얼굴의 굴곡에 따은 음영을 기반으로 검출하기에 조명의 영향을 많이 받음

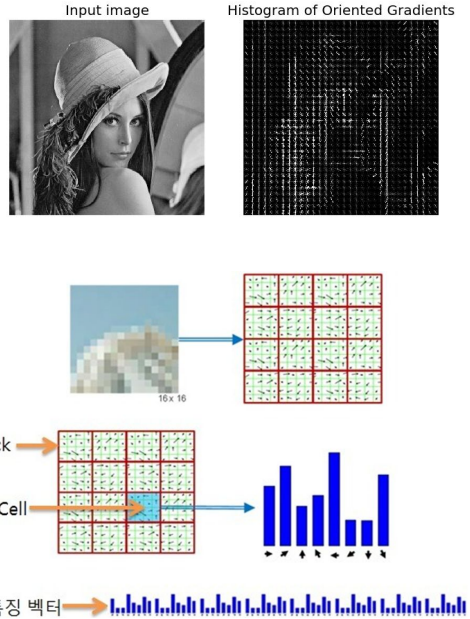


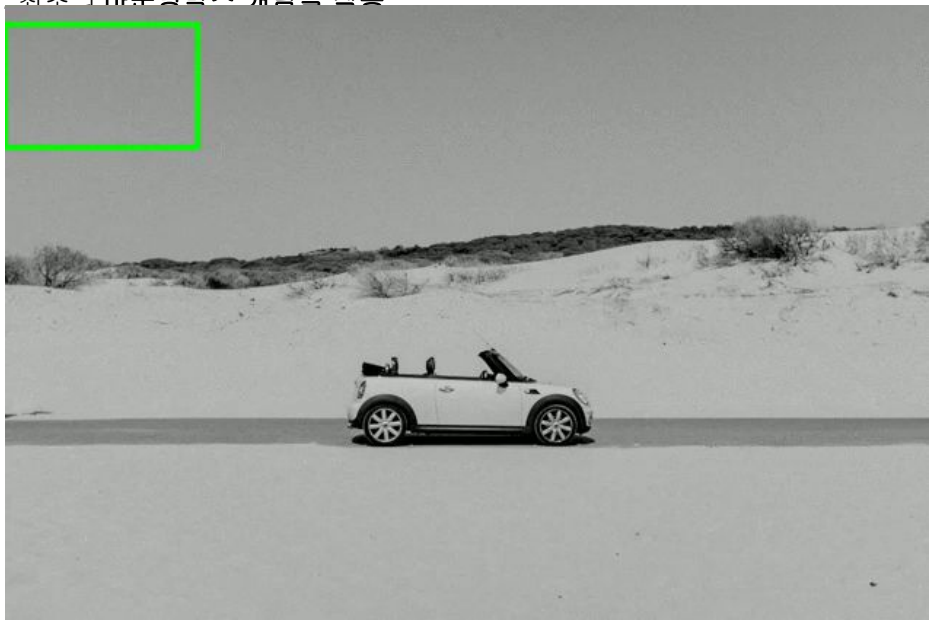
HOG

Histograms of Oriented Gradients

바운딩 박스를 그려 일정 크기의 픽셀마다 이동하며, 박스 영역 안의 block 마다 hog feature에 대해 히스토그램을 분석함으로 해당 block의 특징들을 선별

Edge를 기반으로 검출하지 않기에 조명의 영향을 적게 받음 최초의 **바운딩박스 개념을 적용**

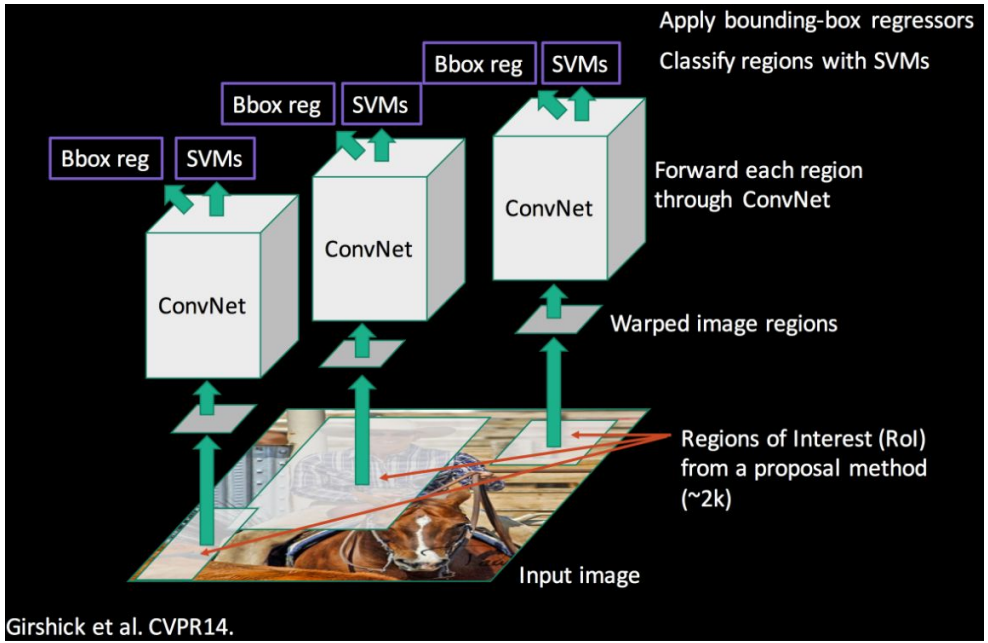


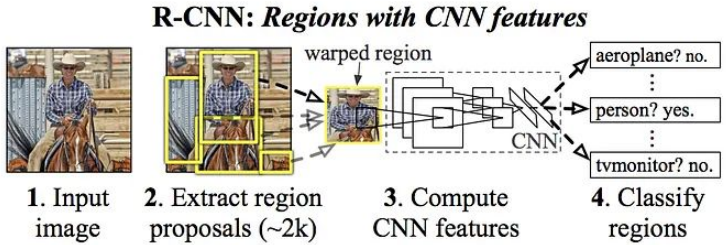


R-CNN

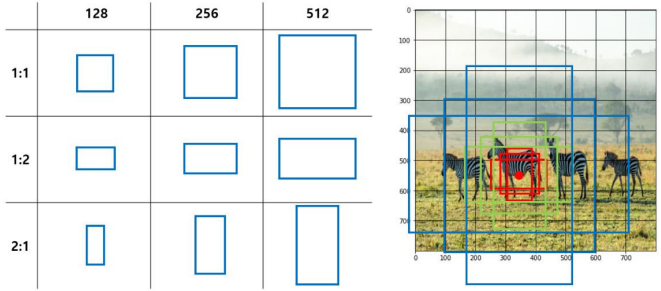
Regin With Convolution Neural Networks features

설정한 region을 CNN의 feature(입력값)로 활용하여 Object detection을 수행하는 신경망이라는 의미 . **2 Stage detector**





anchor box



2 Stage Detector

객체가 있을 법한 위치를 먼저 제안하는 '**물체 후보 영역 추출(Region Proposal)**'과
추출된 영역을 구체적으로 식별·조정하는 '**분류 및 회귀(Classification & Regression)**' 과정을 순차적으로 수행하기 때문에 2단계라고 부름.

즉, 1단계에서 완벽한 Bounding Box를 결정하는 것이 아니라 후보 영역만 뽑아냄.

1단계 (Stage 1): Region Proposal (물체 후보 영역 추출)
• 역할: 이미지 전체를 무작정 조사하는 대신, "이 구역에 무언가 물체가 있을 것 같다"라고 판단되는 수천 개의 후보 영역(Bounding Box의 초기 후보)을 찾아냄.
• 사용 기술: 초기 모델(R-CNN 등)에서는 전통적인 컴퓨터 비전 알고리즘인 Selective Search를 사용했고, 이후 발전된 모델(Faster R-CNN 등)에서는 신경망 기반의 RPN(Region Proposal Network)을 활용

2단계 (Stage 2): Classification & Bounding Box Regression (분류 및 미세 조정)
• 역할: 1단계에서 넘어온 후보 영역들을 대상으로 두 가지 작업을 동시에 수행
1. Classification (분류): 이 구역에 있는 물체가 실제로 무엇인지(사람, 자동차, 고양이 등, 혹은 배경인지) 확률을 계산
2. Bounding Box Regression (박스 위치 미세 조정): 1단계에서 대략적으로 잡아둔 후보 박스의 위치와 크기를 물체에 딱 맞게 정밀하게 다듬는 작업을 처리

1-Stage Detector와의 직관적인 비교
• 2-Stage (예: Faster R-CNN): "여기 물체가 있을 것 같은데?(1단계) → 확인해 보니 '고양이'고 박스 크기는 조금 더 늘려야겠네!(2단계)" 처럼 차근차근 확인하므로 정확도가 높지만 속도가 느림.
• 1-Stage (예: YOLO, SSD): 이미지 전체를 바둑판(Grid)처럼 나눈 뒤, 영역 추출 과정 없이 "어디에 뭐가 있는지" 한 번에 바로 예측. 따라서 속도가 매우 빨라 실시간 탐지에 적합하지만 정확도는 상대적으로 낮을 수 있음.

현업에서 Two-Stage 는 잘 사용하지 않음

YOLO

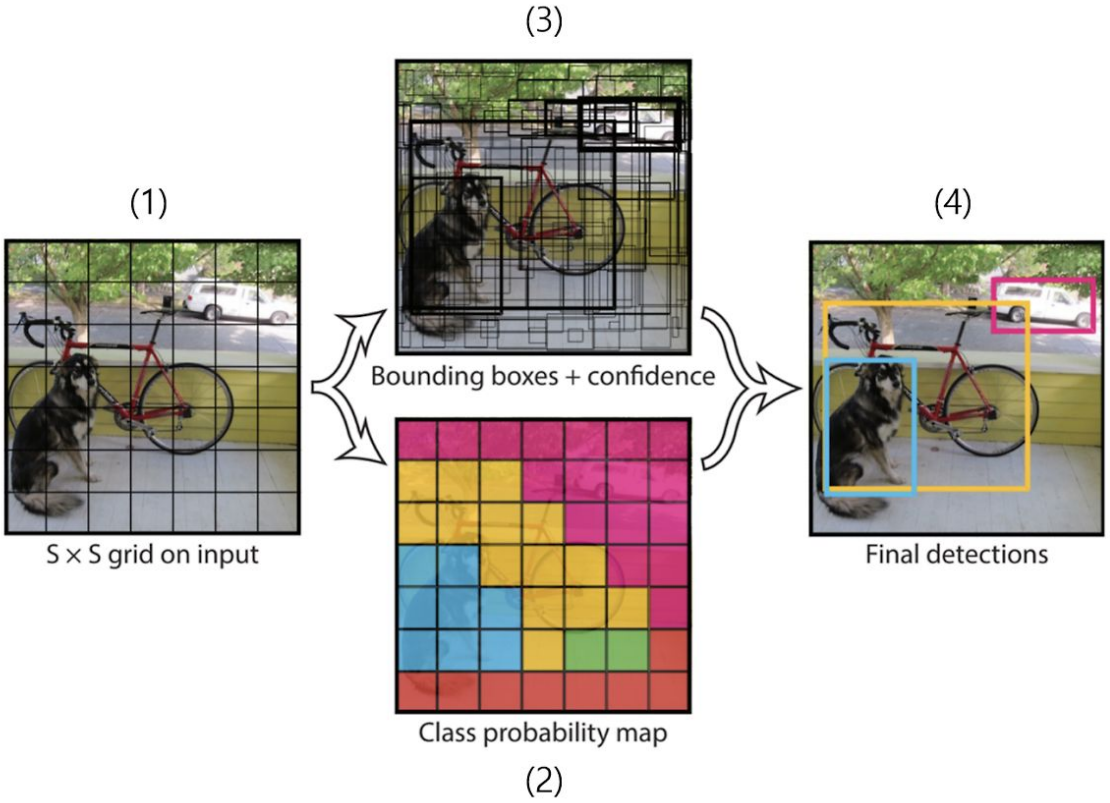
You Only Look Once
조셉 레드몬에 의해 2015년 등장. 컨셉은 real time object detection 즉, 실시간 객체 검출
지난 시간에 설명한 2 stage 계열 같은 경우 객체를 검출하는데 시간이 조금 걸림.
자율주행 자동차와 같은 실시간으로 사물이 어디에 위치하는지 파악되어야 하는 작업에서는 사용될 수 없음.

2 stage detection 방식의 느린 속도 때문에 이둘을 한 번에 하는 **1 stage detection** 방식이 나오게 되며 , 그 시초 모델이 YOLO
YOLO는 classification 과 Localization을 동시에 하여 실시간으로 객체의 검출이 가능

YOLO 특징

- region proposal, feature extraction, classification, bbox regression ⇒ one-stage detection로 통합하여 빠르다
- 주변 정보까지 학습하여 이미지 전체를 처리하기 때문에 background error가 적다
- 훈련 단계에서 보지 못한 새로운 이미지에 대해서도 검출 정확도가 높다

YOLO 동작과정



먼저 사진이 입력되면 가로 세로를 동일한 그리드 영역으로 나눈다.

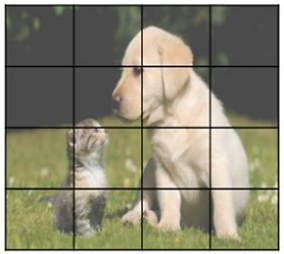
그 후 각 그리드 영역에 대해서 어디에 사물이 존재하는지 바운딩박스와 박스에 대한 신뢰도 점수를 예측한다. 신뢰도가 높을수록 굵게 박스를 그려 준다. 이와 동시에 (2)에서와 같이 어떤 사물인지에 대한 classification작업이 동시에 진행된다.

그러면 굵은 박스들만 남기고 얇은 것들 즉, 사물이 있을 확률이 낮은 것들은 지워 준다.

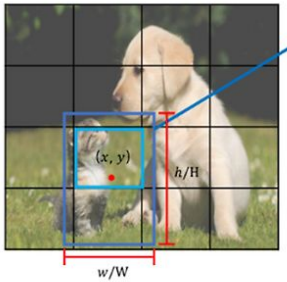
최종 경계박스들을 NMS(Non- Maximum Suppression) 알고리즘을 이용해 선별하면 (4) 이미지처럼 3개만 남게 된다.

YOLO 동작과정 예시

예시) S = 4, B = 2, C = 20



Resized image 를
4x4 grid 로 분할



Grid cell 마다 bbox 2개씩 예측

bbox #1

x
y
w
h
p_c

bbox의 중심좌표의 위치
(grid cell 기준)

Input image W, H 로 normalize

p_c: $\Pr(Object) * IOU_{pred}^{truth}$

cf. $\Pr(Object)$:
물체가 bbox 내에 있으면 1,
없으면 0

이미지를 4x4 그리드로 나눈다 가정하고, (실제 논문에서는 7x7) 이때 각 그리드 셀마다 예측하는 바운딩 박스는 2개이며 전체 고려하는 class 개수는 20개로 가정을 해보자.

첫 번째 예측된 바운딩 박스는 파란색이며 이때 예측된 바운딩 박스의 정 중앙 좌표인 x, y 좌표 그리고 바운딩 박스의 너비와 높이를 전체 이미지로 나눠 노멀라이즈 한 w와 h (0~1 사이의 값을 가짐) 그리고 박스 내부에 물체가 존재할 확률 confidence score인 pc까지 총 5가지 아웃풋이 나오게 된다.

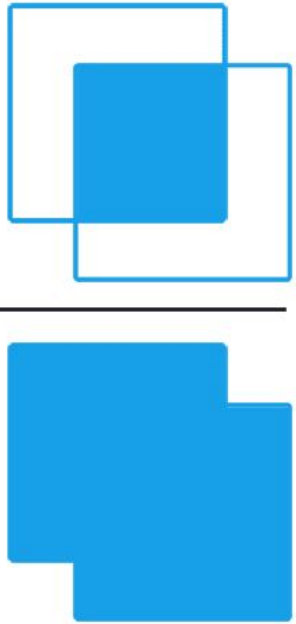
여기서 confidence score인 pc는 pr에 IOU를 곱해준 값이다.
pr은 해당 바운딩 박스 내부에 물체가 있으면 1 없으면 0 값이며, IOU를 곱한값임.

IOU (Intersection Over Union)

IOU: 바운딩 박스의 위치 정확도에 대한 평가 도구

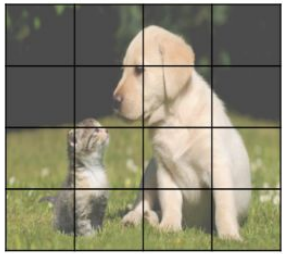
예측한 바운딩 박스와 실제 물체의 바운딩 박스 두 개의 교집합을 총너비로 나눈 값이며 수치가 클수록 정확도가 높은 것임.

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

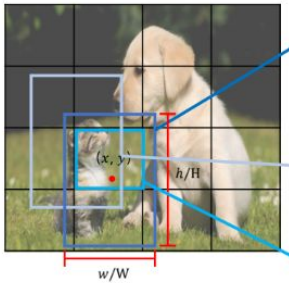


YOLO 동작과정 예시

예시) S = 4, B = 2, C = 20



Resized image 를
4x4 grid 로 분할



Grid cell 마다 bbox 2개씩 예측

bbox #1

bbox #2

x
y
w
h
p_c
x
y
w
h
p_c
c_1
c_2
c_3
⋮
c_{20}

bbox의 중심좌표의 위치
(grid cell 기준)

Input image W, H 로 normalize

p_c: $\Pr(Object) * IOU_{pred}^{truth}$

cf. $\Pr(Object)$:
물체가 bbox 내에 있으면 1,
없으면 0

$\Pr(Class_i|Object)$
: 물체가 bbox 내에 있을 때,
Grid cell에 있는 object가 i번째
class에 속할 확률

위에서 설명했던 과정을 한 번 더 하여 두 번째 바운딩 박스에 대해 예측한 5가지 값에 대해 1차원 텐서로 넣어준다.

마지막으로 하늘색 박스를 보면 그리드 셀에 있는 오브젝트가 어떤 클래스 인지 확률이 들어가게 된다.

이로써 하나의 그리드 셀에 대한 아웃풋이 나오게 되며, 이 과정을 모든 그리드 셀(16개)에 대해 적용시켜 준다.

그러면 이와 같이 4 x 4 x 30의 아웃풋 텐서가 나오게 된다.
(4 x 4 x 30 인 이유는 4 x 4 그리드 셀 각각 마다 5개의 값을 예측하는 2개의 바운딩 박스와 (5*2) 거기에 예시를 들었던 20개의 클래스 일 각각의 확률인 20개가 들어감으로 총 30개 그래서 4 x 4 x 30 이 된다)

YOLO 신경망의 구조 디자인

yolo는 googleNet의 구조로 영감을 받았다고 논문의 저자는 설명했는데, 구글넷의 Inception 블록 대신 단순한 컨볼루션으로 네트워크를 구성했다고 함.
총 24개의 conv layer와 2개의 fc layer로 구성이 되어 있음.
앞의 20개의 conv layer에 대해서는 1000개 클래스의 ImageNet 데이터셋으로 pretrained 된 부분이고 뒤에 4개의 conv layer와 2개의 fc layer를 더 붙여서 Pascal VOC 데이터로 Fine tuning 시킨 과정을 거쳤으며 노란색으로 표시된 중간에 1x1 reduction layer로 연산량을 감소시킨 특징도 있음.

